

LAB03:

Alunos: Gustavo Santiago Sousa

Resumo sobre o Guia da Synopsys

EX:1

```
`timescale 1ns/1ps

module crossbar_structural_tb;

    reg in1;
    reg in2;
    reg select;

    wire out1;
    wire out2;

    crossbar_structural dut (
        .in1(in1),
        .in2(in2),
        .select(select),
        .out1(out1),
        .out2(out2)
    );

    initial begin
        $monitor("select=%b | in1=%b | in2=%b | out1=%b | out2=%b",
            select, in1, in2, out1, out2);

        in1 = 0; in2 = 1; select = 0; #10;

        in1 = 1; in2 = 0; select = 0; #10;

        in1 = 0; in2 = 1; select = 1; #10;

        in1 = 1; in2 = 0; select = 1; #10;

        $finish;
    end
endmodule
```

Ex2:

A principal vantagem é a divisão de responsabilidades, módulos menores podem focar em coisas específicas, enquanto módulos maiores apenas focam na integração dessas funcionalidades. Essa abordagem favorece ainda a manutenção, onde em caso de problema, idealmente o único módulo que precisa ser modificado é o que implementa a funcionalidade. Essa técnica além de aumentar a abstração do sistema, a depender da dependência de dados, pode ser usada para escalar o sistema, instanciando módulos em paralelo.

Ex3:

```
module counter_estrutural (
    input clk_i,
    input rst_i,
    output [23:0] counter_o
);

wire [11:0] counter_lower, counter_higher;

counter #(.COUNTER_WIDTH(12)) counter_inst1(
    .clk(clk_i),
    .rst(rst_i),
    .en_n(1'b1),
    .counter_o(counter_lower)
);

counter #(.COUNTER_WIDTH(12)) counter_inst2(
    .clk(counter_lower[11]),
    .rst(rst_i),
    .en_n(1'b1),
    .counter_o(counter_higher)
);

assign counter_o = {counter_higher, counter_lower};

endmodule
```

Ex: 4

```
module top
#(
    parameter COUNTER_MAX = 16,
    parameter COUNTER_WIDTH = $clog2(COUNTER_MAX)
) (
    input clk_i,
```

```
    input  rst_i,
    input [COUNTER_WIDTH-1:0] default_value,
    output reg out
);

wire [COUNTER_WIDTH-1:0] counter;
wire en;

counter #(.COUNTER_WIDTH(COUNTER_WIDTH)) counter_inst(
    .clk(clk_i),
    .rst(rst_i),
    .en_n(!en),
    .counter_o(counter)
);

compare #(.COUNTER_WIDTH(COUNTER_WIDTH)) compare_inst
(
    .default_value(default_value),
    .counter_i(counter),
    .isEqual(en)
);

always @(posedge clk_i) begin
    out <= en;
end

endmodule
```